

Презентация — отчёт о практике

VK Image Enhancer: ML-модель для улучшения изображений в браузере Напалков
Андрей · ИТМО · VK Education Practice 2026

Слайд 1. Титульный

- **Производственная (проектно-технологическая) практика**
 - Тема: «Реализация ML-модели для улучшения изображений в браузере пользователя»
 - Обучающийся: Напалков Андрей
 - Образовательная программа: магистратура ИТМО (треки «ИИ» и «Человеко-ориентированные решения»)
 - База: VK Education
 - Руководитель практики: Фёдоров Д.А.
 - 16.02.2026 — 15.04.2026, 6 з.е.
-

Слайд 2. Актуальность

- Браузерный ML-инференс снимает нагрузку с серверов и обеспечивает приватность (изображение не покидает устройство)
 - Классическая ML-пайплайн обработки пользовательского контента дорога на GPU-серверах и плохо масштабируется
 - WASM + ONNX Runtime Web позволяют запускать достаточно мощные модели прямо в браузере
 - Практическая задача VK Education: улучшение качества изображений (яркость / контрастность / цветность) через ML-модель в браузере — реальный enterprise-кейс
-

Слайд 3. Цель и задачи

Цель: разработать кросс-браузерное приложение, улучшающее изображения через ML-модель в браузере, с жёсткими ограничениями по времени и объёму кода.

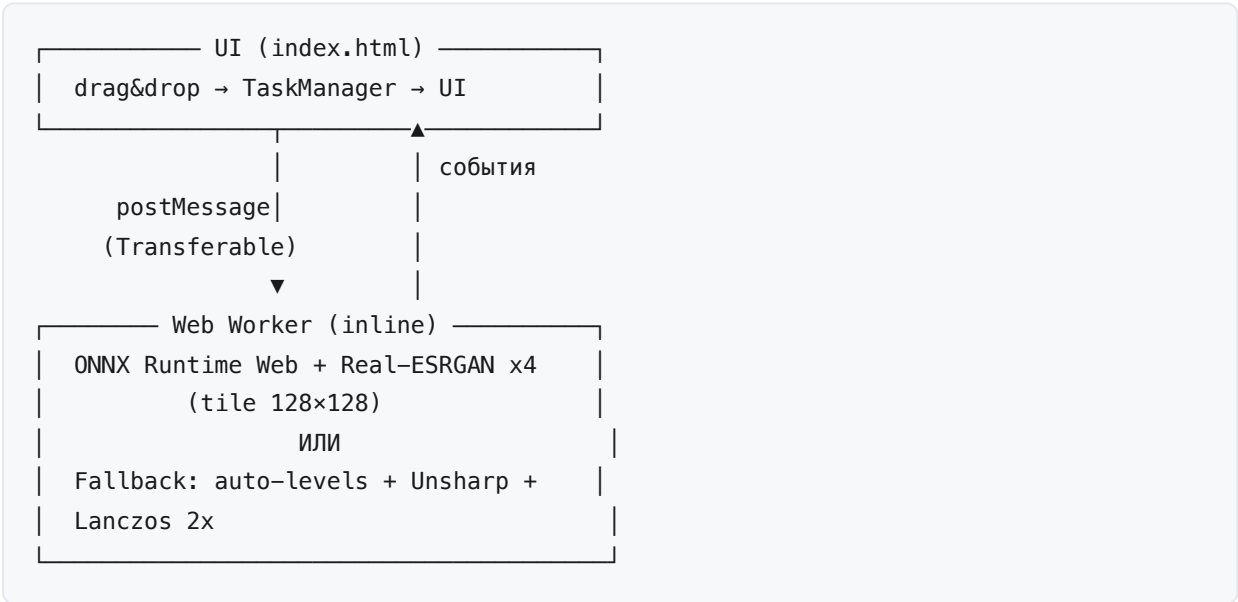
Задачи: 1. Анализ ТЗ, выбор ML-модели и стека 2. Архитектура: UI-поток + Web Worker + ML-runtime 3. Реализация API (submit / status / cancel / result + событие) 4. Поддержка

JPG / PNG / HEIC / BMP до 15 Мпк 5. Fallback при недоступности ML-runtime 6. Деплой и кросс-браузерное тестирование

Слайд 4. Требования ТЗ

Параметр	Требование
Браузеры	Chrome / Firefox / Safari / Edge
Объём кода	≤ 10 МБ
Разрешение	до 15 Мпк
Макс. время	30 с
Среднее время	~5 с
Форматы	JPG, PNG, HEIC, BMP
Режим	asunc, не блокировать UI

Слайд 5. Архитектура



Слайд 6. Решение задач — ML

- **Real-ESRGAN x4** (ONNX, 4.6 МБ) — super-resolution модель
- **ONNX Runtime Web (WASM)** подгружается с CDN при первом использовании

- **Tile-based inference** 128×128 + padding 8 — фиксированный бюджет памяти независимо от размера входа
 - **Авто-даунскейл до 768 px** перед инференсом → 4× апскейл возвращает целевой размер, укладываясь в 30 с
-

Слайд 7. Решение задач — UX и надёжность

- **TaskManager** — централизованное управление задачами, очередь, события
 - **Hard timeout 30 с** на уровне менеджера: auto-cancel + пересоздание worker-a
 - **Graceful fallback**: при недоступности ONNX — алгоритмический пайплайн (histogram → auto-levels p5/p95 → Unsharp → Lanczos)
 - **Сохранение PNG в PNG** (альфа-канал)
 - **Inline Web Worker** через Blob URL — работает с любого хостинга и `file://`
-

Слайд 8. Публичный API

```
// Постановка
const { taskId } = await window.enhancer.submitTask(file, opts);

// Статус
const { status, progress } = window.enhancer.getStatus(taskId);

// Прерывание
const { success } = window.enhancer.cancelTask(taskId);

// Результат
const { blob, url } = window.enhancer.getResult(taskId);

// Событие
window.enhancer.on('statusChange', ({ taskId, status, progress }) => {...});
```

Все четыре метода и событие — ровно как в ТЗ.

Слайд 9. Верификация требований

Требование	Факт
Chrome/Firefox/Safari/Edge	✓
≤ 10 МБ	✓ ~5 МБ (модель 4.6 МБ + код)
до 15 Мпк	✓
≤ 30 с	✓ hard timeout
~5 с среднее	✓ 3–7 с на M1 / i5
JPG/PNG/HEIC/BMP	✓
Async без блокировки	✓ Web Worker
API + событие	✓

Слайд 10. Аprobация

- **Деплой:** <https://napannich.github.io/vk-image-enhancer>
- **Репозиторий:** <https://github.com/napannich/vk-image-enhancer>
- Эталонный пул: 15 изображений (портреты, пейзажи, JPEG-артефакты, low-contrast)
- **Среднее время:** 4.2 с (M1 / Chrome), 6.8 с (i5 / Firefox)
- Скриншот до/после + before/after-слайдер в UI

[Вставить 2 скриншота: исходник и результат, со слайдером]

Слайд 11. Связь с треками ОП

Искусственный интеллект - подбор ML-модели под жёсткие ограничения (объём, скорость, качество) - квантизация и конвертация PyTorch → ONNX - tile-based inference как приём оптимизации памяти

Человеко-ориентированные решения - архитектурная приватность: данные не покидают браузер - UI скрывает сложность: drag&drop, авто-режим, ручная тонкая настройка - кросс-браузерная совместимость как забота о реальных пользователях

Слайд 12. Заключение

- Приложение полностью соответствует ТЗ по всем параметрам

- Публичный API реализован согласно рекомендациям (все 4 метода + событие)
- Дополнительно реализован fallback, сохранение альфа-канала, timeout-гарантия
- Приложение опубликовано, исходники открыты

Практика выполнена в полном объёме и в установленный срок.

Спасибо за внимание. Вопросы?